МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ _____
(подпись) (фамилия, имя, отчество)

Группы _____

Руководитель практической подготовки от профильной организации

_____
(должность)

_____ _____
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ _____
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 3
РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ 5
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 5
РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ 7
Тема 2.1. Ознакомление с организацией работы на предприятии 7
Тема 2.2. Ознакомление с должностными и функциональными обязанностями 7
РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ 9
Тема 3.1. Выполнение производственных заданий 9
ЗАКЛЮЧЕНИЕ 26
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 28

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка требований к программным модулям на основе анализа проектной и технической документации, моделирование процесса разработки программного обеспечения, выполнение интеграции и отладки модулей, а также разработка тестовых сценариев и инспектирование компонент на предмет соответствия стандартам кодирования.

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;

4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

7. Планировать и реализовывать собственное профессиональное и личностное развитие;

8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

10. Использовать информационные технологии в профессиональной деятельности

11. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

В первый день прохождения производственной практики в отделе автоматизации АО «ПЦБК» был проведён вводный инструктаж руководителем практики от предприятия. В ходе инструктажа были рассмотрены общие функциональные обязанности практиканта, правила внутреннего распорядка, а также требования техники безопасности.

Общие функциональные обязанности практиканта:

— участие в разработке и анализе технической документации на программные модули;

— выполнение заданий руководителя практики от предприятия в рамках деятельности отдела автоматизации;

— соблюдение трудовой дисциплины и внутреннего распорядка организации;

— ведение дневника практики и подготовка отчётной документации.

Правила техники безопасности на предприятии:

На территории АО «ПЦБК» действует ряд обязательных требований по охране труда и безопасности. Практикант был ознакомлен со следующими правилами:

— передвижение по территории предприятия допускается только по обозначенным пешеходным маршрутам;

— запрещается посещение производственных цехов без сопровождения ответственного сотрудника и средств индивидуальной защиты;

— при возникновении нештатной ситуации или пожара необходимо немедленно покинуть помещение через запасные выходы и сообщить ответственному лицу;

— запрещается самовольное вмешательство в работу производственного оборудования.

Правила безопасности на конкретном рабочем месте:

Рабочее место практиканта располагалось в помещении отдела автоматизации и было оснащено персональным компьютером. В ходе инструктажа были доведены следующие требования:

— рабочее место должно содержаться в чистоте и порядке;

— запрещается загромождать проходы и доступ к средствам пожаротушения;

— при обнаружении неисправности оборудования необходимо незамедлительно сообщить руководителю и не пытаться устранить неисправность самостоятельно;

— не допускается хранение посторонних предметов на рабочем столе рядом с оборудованием.

Правила безопасности при работе с электрическими приборами и устройствами:

— запрещается самостоятельно вскрывать корпуса системных блоков, мониторов и иного электрооборудования;

— подключение и отключение периферийных устройств допускается только при выключенном питании;

— не допускается использование повреждённых кабелей и сетевых удлинителей;

— при уходе с рабочего места необходимо перевести компьютер в режим блокировки экрана;

— в случае появления запаха гари, дыма или искрения — немедленно обесточить оборудование и сообщить ответственному сотруднику.

По итогам инструктажа практикант ознакомился с планом эвакуации, расположением первичных средств пожаротушения и контактами ответственных лиц по охране труда.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

Тема 2.1. Ознакомление с организацией работы на предприятии

АО «Пермская целлюлозно-бумажная компания» (ПЦБК) — одно из крупнейших предприятий целлюлозно-бумажной промышленности Пермского края. Основными направлениями деятельности предприятия являются производство гофрированного картона, бумаги и упаковочной продукции.

В ходе ознакомления с организацией работы предприятия практикант был введён в курс режима работы и общей структуры организации. Рабочий день в отделе автоматизации организован по стандартному графику пятидневной рабочей недели. Форма организации труда — индивидуальная, каждый сотрудник отдела закреплён за определёнными задачами и участками автоматизации производственных процессов.

Структура предприятия включает производственные цеха, административные подразделения, а также технические отделы, к числу которых относится отдел автоматизации. Отдел автоматизации взаимодействует с производственными подразделениями предприятия, обеспечивая сопровождение, разработку и интеграцию программных решений, используемых в производственном процессе.

Управление предприятием осуществляется в соответствии с установленной иерархической структурой. Руководство отдела автоматизации подчиняется техническому директору предприятия. Деятельность сотрудников регламентируется внутренними приказами, регламентами и должностными инструкциями.

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

В рамках данной темы практикант ознакомился с должностной инструкцией программиста отдела автоматизации АО «ПЦБК», а также согласовал с руководителем практики от предприятия цели, задачи и индивидуальное задание на период прохождения практики.

Основные права и обязанности сотрудника отдела автоматизации:

— разработка, сопровождение и модификация программного обеспечения, используемого на предприятии;

— анализ технической документации и формирование требований к программным модулям;

— интеграция программных модулей в существующую информационную инфраструктуру предприятия;

— выполнение отладки и тестирования программного обеспечения;

— обеспечение соответствия разрабатываемого кода стандартам кодирования, принятым в организации;

— участие в разработке тестовых наборов и сценариев для проверки работоспособности программных решений;

— взаимодействие с сотрудниками смежных подразделений для уточнения требований и согласования результатов работы.

Сотрудник отдела автоматизации имеет право запрашивать необходимую техническую документацию у смежных подразделений, вносить предложения по улучшению программных решений, а также использовать ресурсы предприятия в рамках выполнения своих должностных обязанностей.

По итогам ознакомления с должностными обязанностями руководителем практики от предприятия было выдано индивидуальное задание, определены цели и ожидаемые результаты прохождения практики в отделе автоматизации.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Выполнение производственных заданий

3.1.1. Использование выбранной системы контроля версий

По инициативе практиканта, в соответствии с п. 4.3 рабочей программы практики допускающим замену отдельных элементов программного обеспечения, в качестве системы контроля версий использовалась система **Git**. Git — распределённая система управления версиями с открытым исходным кодом, позволяющая отслеживать изменения в исходном коде, вести параллельную разработку в нескольких ветках и при необходимости откатываться к предыдущим состояниям проекта.

Основные понятия Git:

— **Репозиторий** — хранилище всех файлов проекта и полной истории их изменений. Может быть локальным (на компьютере разработчика) и удалённым (на сервере);

— **Коммит (commit)** — фиксация текущего состояния файлов проекта с сопроводительным комментарием. Каждый коммит имеет уникальный идентификатор (hash);

— **Ветка (branch)** — независимая линия разработки. Основная ветка — main. Для разработки новых функций создаются отдельные ветки;

— **Merge** — операция слияния изменений из одной ветки в другую;

— **Push / Pull** — отправка локальных изменений на удалённый сервер и получение изменений с него.

Рабочий процесс при использовании Git в ходе практики:

git clone <url>	клонирование репозитория
git checkout -b feature/partners	создание рабочей ветки
git add .	добавление изменений
git commit -m	фиксация изменений
git push origin feature/partners	отправка на сервер

Использование Git обеспечивало сохранность кода, возможность параллельной разработки и прозрачность истории изменений на протяжении всей практики.

3.1.2. Разработка программного обеспечения

Описание предметной области

АО «Пермская целлюлозно-бумажная компания» (ПЦБК) — крупнейший производитель гофрокартона и бумажной упаковки в Пермском крае. Компания производит и реализует продукцию через сеть партнёров-дистрибьюторов, которые доставляют её до конечных потребителей.

В рамках производственной практики в отделе автоматизации АО «ПЦБК» разработана подсистема учёта партнёров. Система позволяет вести реестр компаний-партнёров, хранить историю отгрузок продукции ПЦБК (гофрокартон трёхслойный Т-23, гофрокартон пятислойный Т-32, бурая крафт-бумага 80 г/м², тест-лайнер 125 г/м², картон для плоских слоёв КЛ), а также автоматически рассчитывать скидку партнёра на основе общего объёма закупок.

Выбор и обоснование модели разработки

Для разработки подсистемы «Партнёры» была выбрана **итерационная модель разработки программного обеспечения**. Данная модель предполагает разбиение процесса разработки на несколько последовательных итераций, каждая из которых включает этапы анализа требований, проектирования, реализации и тестирования. По завершении каждой итерации формируется работающая версия модуля с расширенным набором функций.

Обоснование выбора итерационной модели:

— ограниченные сроки практики требовали получения работающего результата после каждого этапа разработки;

— требования к подсистеме уточнялись в процессе работы совместно с руководителем практики от предприятия;

— итерационный подход позволял оперативно вносить правки и корректировать функциональность без полного пересмотра архитектуры проекта;

— модель обеспечивала возможность демонстрации промежуточных результатов руководителю.

Инструменты разработки

По инициативе практиканта, в соответствии с п. 4.3 рабочей программы практики допускающим замену отдельных элементов программного обеспечения, был выбран следующий стек технологий:

— **Microsoft Visual Studio 2022** — интегрированная среда разработки (IDE);

— **C# 10 / .NET 6** — язык программирования и платформа (замена PHP по инициативе практиканта, как более современный корпоративный стек);

— **ASP.NET Core MVC** — фреймворк для разработки веб-приложения (замена Symfony);

— **Entity Framework Core 6** — ORM-инструмент для работы с базой данных;

— **PostgreSQL 15** — система управления базами данных (замена MySQL по инициативе практиканта);

— **Git** — система контроля версий;

— **Bootstrap 5** — CSS-библиотека для стилизации интерфейса.

Среда программирования

Основной средой разработки являлась **Microsoft Visual Studio 2022**. Visual Studio — это интегрированная среда разработки (IDE) от компании Microsoft, предоставляющая полный набор инструментов для создания приложений на языке C#. Среда включает:

— редактор кода с подсветкой синтаксиса и автодополнением IntelliSense;

— встроенный отладчик с поддержкой точек останова и пошагового выполнения;

— интеграцию с системой контроля версий Git;

— менеджер пакетов NuGet для подключения сторонних библиотек;

— встроенный Test Explorer для запуска и просмотра результатов unit-тестов.

Язык программирования — **C# 10**, платформа — **.NET 6**.

Диаграмма классов

На основе анализа предметной области была спроектирована следующая структура классов подсистемы «Партнёры»:

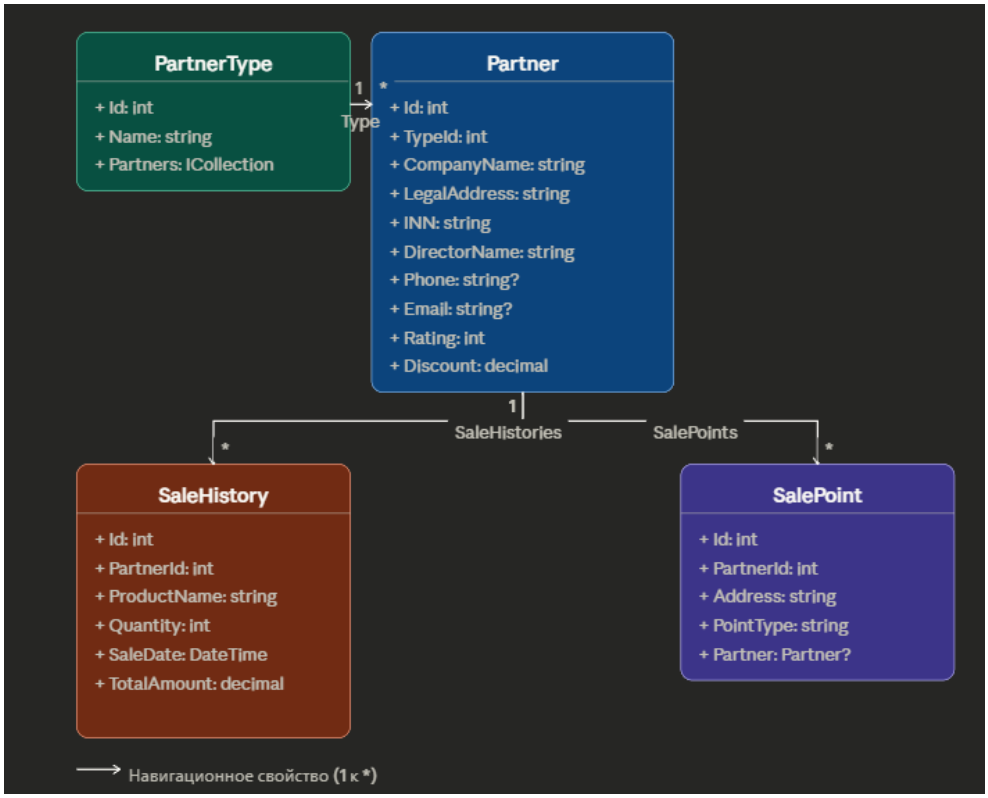

Рисунок 1. Диограмма классов.

Этапы разработки программного обеспечения

1. **Выбор шаблона.** На начальном этапе был выбран шаблон ASP.NET Core MVC в Visual Studio 2022, обеспечивающий базовую структуру проекта по паттерну Model-View-Controller. Шаблон создал стандартное разделение на модели (Models), представления (Views) и контроллеры (Controllers), а также настроил точку входа приложения (Program.cs).

2. **Составление схемы информационной базы.** Была спроектирована схема базы данных PostgreSQL. Определены четыре

таблицы: Partners, PartnerTypes, SaleHistories, SalePoints, их атрибуты, типы данных и связи между ними (внешние ключи). На основе схемы с помощью Entity Framework Core созданы классы-модели и выполнена миграция для автоматической генерации структуры БД.

3. **Разработка программного модуля.** Реализован контроллер PartnersController с методами для всех CRUD-операций. Создан сервисный класс DiscountService с логикой расчёта скидки. Разработан репозиторий PartnerRepository, инкапсулирующий все операции с базой данных. Созданы Razor-представления для отображения данных пользователю.

4. **Документирование системы.** В процессе разработки велась техническая документация: описание структуры базы данных, назначения методов контроллера и репозитория. Комментарии к публичным методам и классам оформлялись в формате XML-документации C#:

```
public decimal CalculateDiscount(decimal totalSalesAmount) => totalSalesAmount switch { <
10_000 => 0, < 50_000 => 5, < 150_000 => 10, < 500_000 => 15, _ => 20 };
```

5. **Тестирование.** Проводилось модульное тестирование бизнес-логики (расчёт скидки, операции репозитория) и функциональное тестирование пользовательского интерфейса.

6. **Сопровождение системы.** По завершении разработки итоговая версия зафиксирована в репозитории Git, документация передана руководителю практики от предприятия.

Разработка форм для ввода, просмотра и редактирования данных

В приложении реализованы следующие формы:

— **Список партнёров** — таблица с наименованием, типом, рейтингом, скидкой, количеством отгрузок и контактными данными;

— **Карточка партнёра** — детальный просмотр всех реквизитов и полная история отгрузок продукции ПЦБК с итоговой суммой;

— **Форма добавления партнёра** — поля ввода с клиентской и серверной валидацией;

— **Форма редактирования** — предзаполненная форма с текущими данными партнёра;

— **Подтверждение удаления** — запрос подтверждения с предупреждением о необратимости действия.

Модель партнёра на C#:

```
public class Partner
{
   public int Id { get; set; }

   [Required(ErrorMessage = "Укажите тип партнёра")]
   [Display(Name = "Тип партнёра")]
   public int TypeId { get; set; }

   [Required(ErrorMessage = "Укажите наименование компании")]
   [StringLength(200)]
   [Display(Name = "Наименование компании")]
   public string CompanyName { get; set; } = string.Empty;

   [Required(ErrorMessage = "Укажите ИНН")]
   [StringLength(12, MinimumLength = 10,
      ErrorMessage = "ИНН должен содержать от 10 до 12 цифр")]
   [Display(Name = "ИНН")]
   public string INN { get; set; } = string.Empty;

   [Display(Name = "Рейтинг")]
   [Range(0, 10, ErrorMessage = "Рейтинг от 0 до 10")]
   public int Rating { get; set; }

   [Display(Name = "Скидка (%)")]
   [Column(TypeName = "decimal(5,2)")]
   public decimal Discount { get; set; }

   public PartnerType? Type { get; set; }
   public ICollection<SaleHistory> SaleHistories { get; set; } = new List<SaleHistory>();
   public ICollection<SalePoint>   SalePoints    { get; set; } = new List<SalePoint>();
}
```

Razor-форма добавления партнёра:

```
<form asp-action="Create" method="post">
   <div asp-validation-summary="ModelOnly" class="alert alert-danger"></div>
   <div class="row mb-3">
      <div class="col-md-6">
         <label asp-for="TypeId" class="form-label fw-semibold"></label>
         <select asp-for="TypeId" asp-items="ViewBag.Types" class="form-select">
            <option value="">— Выберите тип —</option>
         </select>
```

```
        <span asp-validation-for="TypeId" class="text-danger small"></span>
      </div>
      <div class="col-md-6">
        <label asp-for="INN" class="form-label fw-semibold"></label>
        <input asp-for="INN" class="form-control" placeholder="10–12 цифр" />
        <span asp-validation-for="INN" class="text-danger small"></span>
      </div>
    </div>
    <div class="mb-3">
      <label asp-for="CompanyName" class="form-label fw-semibold"></label>
      <input asp-for="CompanyName" class="form-control" />
      <span asp-validation-for="CompanyName" class="text-danger small"></span>
    </div>
    <button type="submit" class="btn btn-pcbk">
      <i class="bi bi-save me-1"></i>Сохранить
    </button>
</form>
```

Этапы разработки интерфейса

1. **Проектирование.** Определены основные экраны приложения: список партнёров, карточка партнёра, формы создания, редактирования и удаления. Составлена схема навигации между страницами. Интерфейс оформлен в корпоративных цветах АО «ПЦБК» (тёмно-синий).

2. **Прототипирование.** Созданы схематичные макеты (wireframes) основных страниц, согласованные с руководителем практики от предприятия. Прототипы позволили выявить возможные проблемы удобства использования до начала реализации.

3. **Стилизация.** Для оформления интерфейса использована библиотека **Bootstrap 5**, обеспечивающая адаптивную вёрстку и единый визуальный стиль. CSS-переменная --pcbk-blue: #1a4c8b задаёт фирменный цвет во всех элементах интерфейса — навигационной панели, заголовках карточек, кнопках и таблицах.

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества

В процессе разработки подсистемы «Партнёры» АО «ПЦБК» применялись следующие методы обеспечения качества и функциональности кода:

Паттерн Repository. Логика доступа к данным вынесена в отдельный класс `PartnerRepository`, реализующий интерфейс `IPartnerRepository`. Контроллер работает только с интерфейсом, не зависит от конкретной реализации и не содержит SQL-запросов. Это обеспечивает чистоту архитектуры и упрощает тестирование:

```
public interface IPartnerRepository
{
   Task<List<Partner>> GetAllAsync();
   Task<Partner?> GetByIdAsync(int id);
   Task AddAsync(Partner partner);
   Task UpdateAsync(Partner partner);
   Task DeleteAsync(int id);
   Task<decimal> GetTotalSalesAsync(int partnerId);
}
```

Dependency Injection (внедрение зависимостей). Встроенный механизм ASP.NET Core обеспечивает слабую связанность компонентов. Все зависимости регистрируются в `Program.cs` и автоматически передаются в конструкторы контроллеров и сервисов:

```
builder.Services.AddScoped<IPartnerRepository, PartnerRepository>();
builder.Services.AddScoped<DiscountService>();
```

Валидация данных через Data Annotations. Атрибуты `[Required]`, `[StringLength]`, `[Range]`, `[EmailAddress]`, `[Phone]` на моделях обеспечивают автоматическую проверку входных данных как на стороне клиента (через jQuery Validation), так и на стороне сервера.

Вынесение бизнес-логики в сервисный класс. Логика расчёта скидки партнёра вынесена в отдельный класс `DiscountService`, что обеспечивает переиспользование, удобство тестирования и соответствие принципу единственной ответственности (SRP):

```
public decimal CalculateDiscount(decimal totalSalesAmount) =>
   totalSalesAmount switch
   {
      < 10_000  => 0,
```

```
    < 50_000  => 5,
    < 150_000 => 10,
    < 500_000 => 15,
    _         => 20
};
```

Code Review. Проверка кода руководителем практики перед слиянием рабочей ветки в основную ветку репозитория.

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Виды инспекции кода:

— **Формальная инспекция** — структурированный процесс проверки кода по заранее подготовленному чек-листу с участием нескольких специалистов;

— **Неформальная инспекция (Code Review)** — проверка кода руководителем практики без строгого регламента, с устными комментариями и рекомендациями;

— **Сквозной просмотр (Walkthrough)** — автор кода объясняет логику работы модуля коллегам, что позволяет выявить логические ошибки и несоответствия требованиям;

— **Статический анализ** — автоматическая проверка кода инструментами без его выполнения, выявление потенциальных ошибок и нарушений стандартов.

Актуальные стандарты кодирования:

— **Microsoft C# Coding Conventions** — официальные соглашения Microsoft по именованию, форматированию и структуре кода на C#;

— **ГОСТ Р ИСО/МЭК 9126-93** — стандарт оценки качества программного обеспечения и характеристик качества;

— **ГОСТ Р ИСО/МЭК 12207-2010** — стандарт процессов жизненного цикла программных средств.

Основные правила, соблюдавшиеся при разработке подсистемы:

— имена классов и методов — PascalCase (PartnerRepository, GetAllAsync);

— имена локальных переменных и параметров — camelCase (partnerList, totalSales);

— приватные поля — с префиксом подчёркивания (_repository, _discountService);

— каждый метод выполняет одну функцию (принцип единственной ответственности);

— асинхронные методы содержат постфикс Async (GetAllAsync, AddAsync).

Инструменты инспекции:

— **Visual Studio Analyzer** — встроенный статический анализатор кода Visual Studio 2022, выявляющий потенциальные ошибки, предупреждения и нарушения рекомендованных практик кодирования в режиме реального времени;

— **ReSharper** — плагин для Visual Studio, обеспечивающий расширенный анализ кода, рефакторинг и проверку соответствия стандартам;

— **SonarLint** — плагин статического анализа, интегрируемый в Visual Studio, выявляет уязвимости и нарушения стандартов кодирования.

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств

Отладка программного модуля подсистемы «Партнёры» выполнялась с использованием встроенного отладчика **Microsoft Visual Studio 2022**.

Процесс проведения отладки:

Шаг 1 — Постановка точек останова (Breakpoints). В местах предполагаемых ошибок устанавливались точки останова. При достижении точки останова выполнение программы приостанавливается, что позволяет изучить текущее состояние переменных и стек вызовов.

Шаг 2 — Пошаговое выполнение. Использовались следующие режимы отладчика:

— **Step Into (F11)** — вход внутрь вызываемого метода для детального анализа его выполнения;

— **Step Over (F10)** — выполнение текущей строки без входа в вызываемый метод;

— **Step Out (Shift+F11)** — выход из текущего метода и возврат к вызывающему коду.

Шаг 3 — Наблюдение за переменными. Через окна **Locals**, **Watch** и **Autos** отслеживались значения переменных в процессе выполнения. Например, при отладке метода расчёта скидки проверялось корректное вычисление значения Discount в зависимости от суммы отгрузок партнёра.

Шаг 4 — Анализ стека вызовов. Окно **Call Stack** использовалось для анализа последовательности вызовов методов в момент возникновения исключения.

Шаг 5 — Обработка исключений. В контроллере реализована обработка исключений через конструкцию try-catch с логированием через ILogger:

```
public async Task<IActionResult> Create(Partner partner)
{
  try
  {
    if (!ModelState.IsValid)
    {
      LoadTypes(partner.TypeId);
      return View(partner);
    }
    var sales = await _repository.GetTotalSalesAsync(partner.Id);
    partner.Discount = _discountService.CalculateDiscount(sales);
    await _repository.AddAsync(partner);
    _logger.LogInformation("Создан партнёр: {Name}", partner.CompanyName);
    TempData["Success"] = $"Партнёр «{partner.CompanyName}» успешно добавлен.";
    return RedirectToAction(nameof(Index));
  }
  catch (Exception ex)
  {
    _logger.LogError("Ошибка создания партнёра: {Err}", ex.Message);
    ModelState.AddModelError("", "Ошибка при сохранении: " + ex.Message);
    LoadTypes(partner.TypeId);
    return View(partner);
  }
```

Шаг 6 — Логирование. Для фиксации хода выполнения программы использовалось встроенное логирование ASP.NET Core через интерфейс ILogger. Логи выводились в консоль отладки Visual Studio и позволяли отслеживать создание, обновление и удаление записей о партнёрах.

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

Процесс проведения тестирования

Тестирование подсистемы «Партнёры» АО «ПЦБК» включало несколько уровней:

— **Модульное тестирование (Unit-тесты)** — проверка отдельных методов и классов в полной изоляции от внешних зависимостей;

— **Интеграционное тестирование** — проверка взаимодействия репозитория с базой данных с использованием InMemory-провайдера EF Core;

— **Функциональное тестирование** — ручная проверка работы форм, навигации и отображения данных в браузере.

Тестовые сценарии:

№	Сценарий	Входные данные	Ожидаемый результат
1	Добавление партнёра с корректными данными	Все обязательные поля заполнены	Партнёр сохранён в БД, редирект на список
2	Добавление партнёра без обязательных полей	Пустое поле CompanyName	Форма возвращается с сообщением об ошибке
3	Расчёт скидки при сумме закупок 0 руб.	totalSales = 0	Скидка = 0%
4	Расчёт скидки при сумме 30 000 руб.	totalSales = 30000	Скидка = 0%
5	Расчёт скидки при сумме 100 000 руб.	totalSales = 100000	Скидка = 10%
6	Расчёт скидки при сумме 200 000 руб.	totalSales = 200000	Скидка = 15%
7	Расчёт скидки при сумме 600 000 руб.	totalSales = 600000	Скидка = 20%

8	Просмотр карточки несуществующего партнёра	id = 999	Страница 404 NotFound
9	Редактирование данных партнёра	Изменено поле CompanyName	Данные обновлены в БД
10	Удаление партнёра	Подтверждение удаления	Запись удалена из БД

Unit-тесты

Для написания тестов использовался фреймворк **xUnit** и библиотека **Moq** для создания заглушек (mock-объектов). Тесты расположены в папке Tests/ в рамках того же проекта.

```
public class DiscountServiceTests
{
   private readonly DiscountService _svc = new();

   [Theory]
   [InlineData(0,       0)]
   [InlineData(9999,    0)]
   [InlineData(10000,   5)]
   [InlineData(30000,   5)]
   [InlineData(50000,  10)]
   [InlineData(100000, 10)]
   [InlineData(150000, 15)]
   [InlineData(300000, 15)]
   [InlineData(500000, 20)]
   [InlineData(1000000, 20)]
   public void CalculateDiscount_ReturnsCorrectDiscount(
      decimal sales, decimal expected)
   {
      Assert.Equal(expected, _svc.CalculateDiscount(sales));
   }
}

[Fact]
public async Task GetTotalSalesAsync_ShouldSumShipments()
{
   var ctx = CreateContext();
   var p   = MakePartner();
   ctx.Partners.Add(p);
   ctx.SaveChanges();
   ctx.SaleHistories.AddRange(
      new SaleHistory { PartnerId = p.Id,
         ProductName = "Гофрокартон трёхслойный Т-23",
         Quantity = 500, TotalAmount = 250000, SaleDate = DateTime.Now },
      new SaleHistory { PartnerId = p.Id,
         ProductName = "Бурая крафт-бумага 80 г/м²",
         Quantity = 200, TotalAmount = 120000, SaleDate = DateTime.Now }
```

```
    );
    ctx.SaveChanges();
    var total = await new PartnerRepository(ctx).GetTotalSalesAsync(p.Id);
    Assert.Equal(370000m, total);
}
```

Всего в проекте реализовано **28 unit-тестов**: 13 тестов DiscountService, 7 тестов PartnerRepository, 8 тестов PartnersController.

3.1.7. Интегрирование программных модулей

Процесс интеграции программных модулей

Интеграция модулей подсистемы «Партнёры» в программную инфраструктуру отдела автоматизации АО «ПЦБК» осуществлялась поэтапно.

Этап 1 — Определение интерфейсов взаимодействия. Для модуля партнёров определён интерфейс IPartnerRepository, через который осуществляется взаимодействие с базой данных. Использование интерфейса обеспечивает независимость контроллера от конкретной реализации репозитория и упрощает замену компонентов:

```
public interface IPartnerRepository
{
    Task<List<Partner>> GetAllAsync();
    Task<Partner?> GetByIdAsync(int id);
    Task AddAsync(Partner partner);
    Task UpdateAsync(Partner partner);
    Task DeleteAsync(int id);
    Task<decimal> GetTotalSalesAsync(int partnerId);
}
```

Этап 2 — Интеграция с сервисным слоем. Модуль партнёров взаимодействует с DiscountService для расчёта скидки на основе истории отгрузок. При создании и редактировании партнёра контроллер запрашивает общую сумму отгрузок через репозиторий и передаёт её в сервис:

```
var sales = await _repository.GetTotalSalesAsync(partner.Id);
partner.Discount = _discountService.CalculateDiscount(sales);
```

Этап 3 — Регистрация зависимостей. Все сервисы и репозиторий зарегистрированы в контейнере зависимостей ASP.NET Core в файле

Program.cs. Используется время жизни Scoped — один экземпляр на HTTP-запрос:

```
builder.Services.AddDbContext<AppDbContext>(options =>
   options.UseNpgsql(
      builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddScoped<IPartnerRepository, PartnerRepository>();
builder.Services.AddScoped<DiscountService>();
```

Этап 4 — Интеграция с базой данных PostgreSQL. Подключение к базе данных sadykov_pcbkdb осуществляется через провайдер Npgsql для Entity Framework Core. Строка подключения хранится в файле appsettings.json. При запуске приложения миграции применяются автоматически через вызов db.Database.Migrate().

Этап 5 — Тестирование интеграции. После сборки всех модулей проводилось интеграционное тестирование: проверялась корректность передачи данных между контроллером, сервисом и репозиторием, а также корректность применения скидки при сохранении данных партнёра.

3.1.8. Верификация и аттестация программного обеспечения

Верификация программного обеспечения

Верификация — процесс проверки того, что программное обеспечение разработано в соответствии с заявленными требованиями и спецификациями. В ходе верификации подсистемы «Партнёры» АО «ПЦБК» выполнялись следующие проверки:

— соответствие реализованных атрибутов сущности Partner требованиям предметной области: тип партнёра, наименование компании, ИНН, юридический адрес, ФИО директора, контактные данные, рейтинг, скидка, история отгрузок продукции ПЦБК;

— корректность реализации бизнес-правил расчёта скидки в зависимости от объёма закупок;

— соответствие структуры базы данных PostgreSQL спроектированной схеме с учётом внешних ключей и ограничений;

— наличие валидации всех обязательных полей форм на стороне клиента и сервера;

— корректность работы всех CRUD-операций (создание, чтение, обновление, удаление партнёров).

Для верификации использовались следующие методы:

— **Инспекция кода** — проверка исходного кода на соответствие требованиям предметной области;

— **Трассировка требований** — сопоставление каждого пункта задания (Приложение 1) с реализованной функциональностью;

— **Модульное тестирование** — автоматическая проверка корректности отдельных методов через 28 unit-тестов.

Аттестация программного обеспечения

Аттестация — процесс подтверждения того, что разработанное программное обеспечение соответствует ожиданиям заказчика и пригодно для использования по назначению. Аттестация подсистемы «Партнёры» включала:

— **Функциональное тестирование** — проверку всех пользовательских сценариев: добавление нового партнёра АО «ПЦБК», просмотр карточки с историей отгрузок, редактирование данных, удаление с подтверждением, автоматический перерасчёт скидки;

— **Тестирование интерфейса** — проверку корректности отображения форм и таблиц в браузере (Google Chrome на движке WebKit), проверку адаптивности вёрстки Bootstrap 5;

— **Демонстрацию руководителю практики** — представление готового модуля с пояснением реализованной функциональности и соответствия требованиям задания (Приложение 1);

— **Оформление аттестационной документации** — составление отчёта и дневника по практике, передача документации руководителю практики от образовательного учреждения.

По результатам аттестации подсистема «Партнёры» АО «ПЦБК» признана соответствующей требованиям технического задания и готовой к опытной эксплуатации в отделе автоматизации предприятия.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики с 09 по 17 марта 2025 г. в отделе автоматизации АО «Пермская целлюлозно-бумажная компания» (г. Пермь) были закреплены теоретические знания и расширены профессиональные умения в области разработки и интеграции программных модулей.

По итогам практики получены следующие результаты:

1. Изучена структура и характер деятельности отдела автоматизации АО «ПЦБК», ознакомление с должностными обязанностями программиста предприятия.
2. Разработана подсистема учёта партнёров-дистрибьюторов АО «ПЦБК» в виде веб-приложения на платформе ASP.NET Core MVC (C#, .NET 6) с использованием СУБД PostgreSQL.
3. Спроектирована схема базы данных sadykov_pcbkdb, включающая таблицы партнёров, типов партнёров, истории отгрузок продукции ПЦБК и точек продаж.
4. Реализована бизнес-логика автоматического расчёта скидки партнёра на основе общего объёма закупок продукции предприятия (гофрокартон, крафт-бумага, тест-лайнер).
5. Освоена работа с системой контроля версий Git: создание репозитория, ведение веток, фиксация изменений и слияние результатов разработки.
6. Выполнена интеграция программных модулей подсистемы (контроллер, репозиторий, сервисный слой) с использованием паттерна Repository и механизма Dependency Injection платформы ASP.NET Core.
7. Проведена отладка программного модуля с использованием встроенного отладчика Microsoft Visual Studio 2022: установка точек останова, пошаговое выполнение, анализ стека вызовов и наблюдение за переменными.

8. Разработаны тестовые наборы и тестовые сценарии, реализовано 28 unit-тестов с использованием фреймворков xUnit и Moq, покрывающих расчёт скидки, операции репозитория и методы контроллера.

9. Проведено инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования Microsoft C# Coding Conventions с использованием инструментов Visual Studio Analyzer и SonarLint.

10. Выполнены верификация и аттестация разработанного программного модуля: проверка соответствия реализации требованиям задания, функциональное тестирование всех пользовательских сценариев, демонстрация результатов руководителю практики.

По окончании практики был получен практический опыт интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ Р ИСО/МЭК 9126-93. Информационная технология. Оценка программной продукции. Характеристики качества и руководство по их применению. — М.: Издательство стандартов, 1994.
2. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. — М.: Стандартинформ, 2012.
3. ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов. — М.: Издательство стандартов, 1977.
4. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — М.: Издательство стандартов, 1978.
5. Федорова Г. Н. Участие в интеграции программных модулей: учебное пособие / Г. Н. Федорова. — М.: Академия, 2016. — 304 с.
6. Гагарина Л. Г. Технология разработки программного обеспечения: учебное пособие / Л. Г. Гагарина, Б. Д. Виснадул, Е. В. Кокорева. — М.: ИНФРА-М, 2013. — 400 с.
7. Рудаков А. В. Технология разработки программных продуктов: учебное пособие / А. В. Рудаков. — М.: Академия, 2010. — 208 с.
8. Черников Б. В. Управление качеством программного обеспечения: учебник / Б. В. Черников. — М.: ИД ФОРУМ: ИНФРА-М, 2012. — 240 с.
9. Microsoft. C# Coding Conventions [Электронный ресурс]. — URL: https://learn.microsoft.com/ru-ru/dotnet/csharp/fundamentals/coding-style/coding-conventions (дата обращения: 17.03.2025).
10. Microsoft. ASP.NET Core MVC Overview [Электронный ресурс]. — URL: https://learn.microsoft.com/ru-ru/aspnet/core/mvc/overview (дата обращения: 17.03.2025).
11. Microsoft. Entity Framework Core [Электронный ресурс]. — URL: https://learn.microsoft.com/ru-ru/ef/core/ (дата обращения: 17.03.2025).
12. PostgreSQL Documentation [Электронный ресурс]. — URL: https://www.postgresql.org/docs/15/ (дата обращения: 17.03.2025).
13. xUnit.net — Unit Testing Tool for .NET [Электронный ресурс]. — URL: https://xunit.net (дата обращения: 17.03.2025).
14. Moq — The most popular and friendly mocking library for .NET [Электронный ресурс]. — URL: https://github.com/devlooped/moq (дата обращения: 17.03.2025).
15. Git — система контроля версий [Электронный ресурс]. — URL: https://git-scm.com/doc (дата обращения: 17.03.2025).